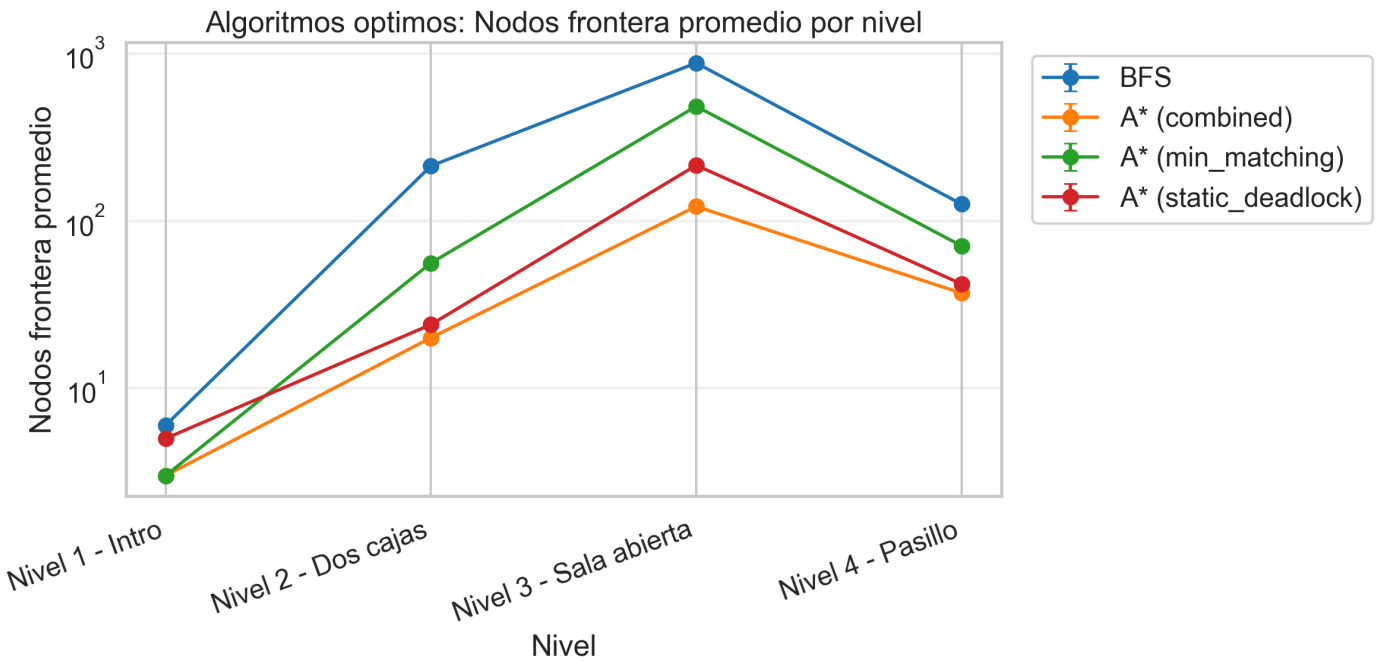
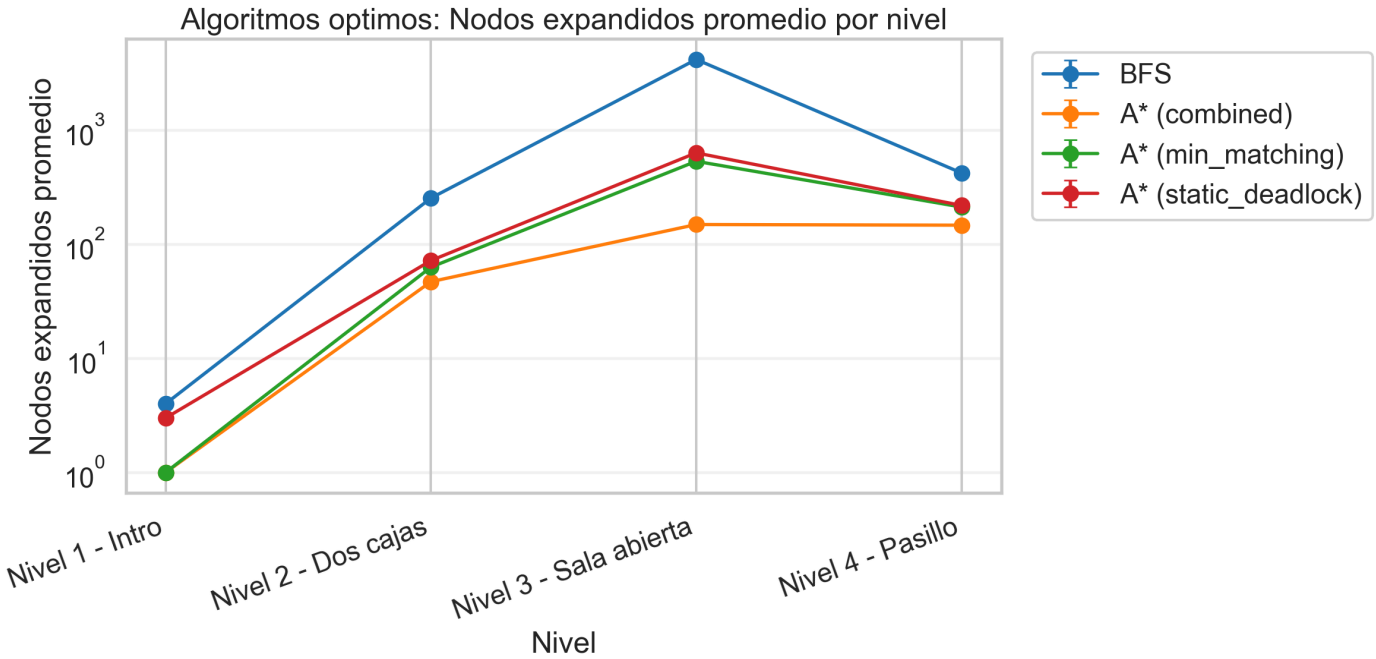
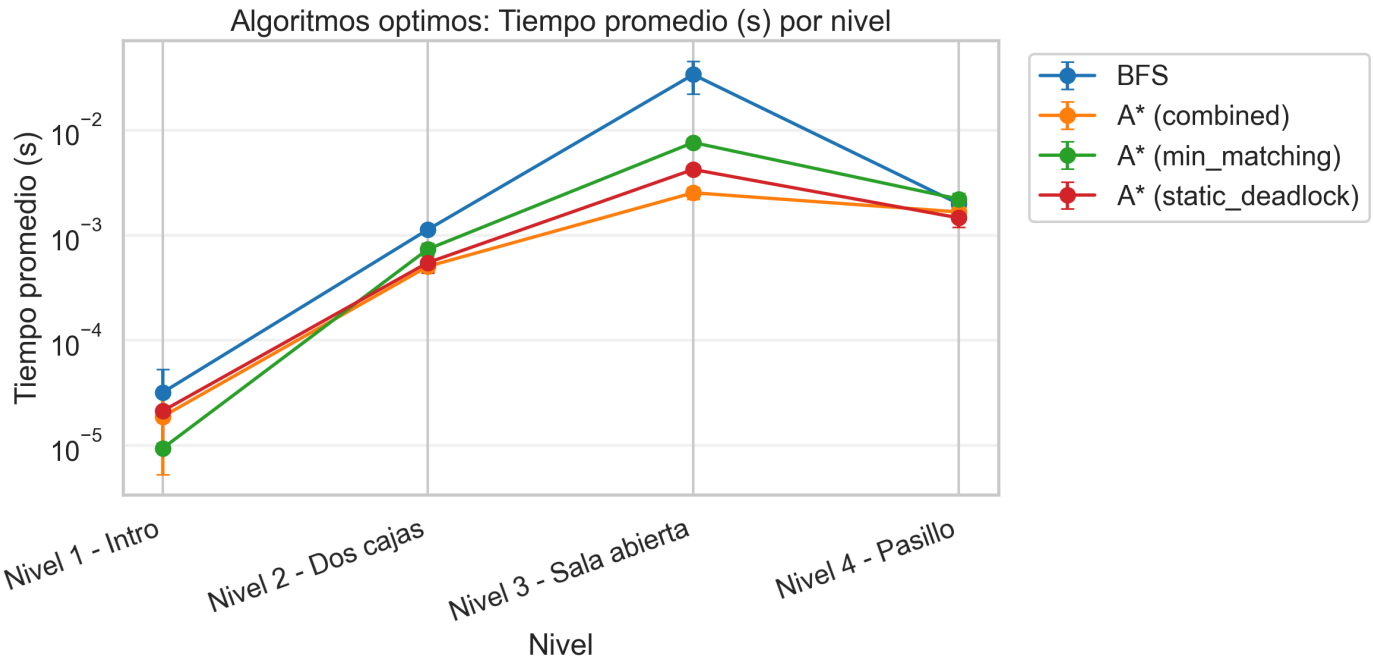
ALGORITMOS OPTIMOS (BFS vs A*)

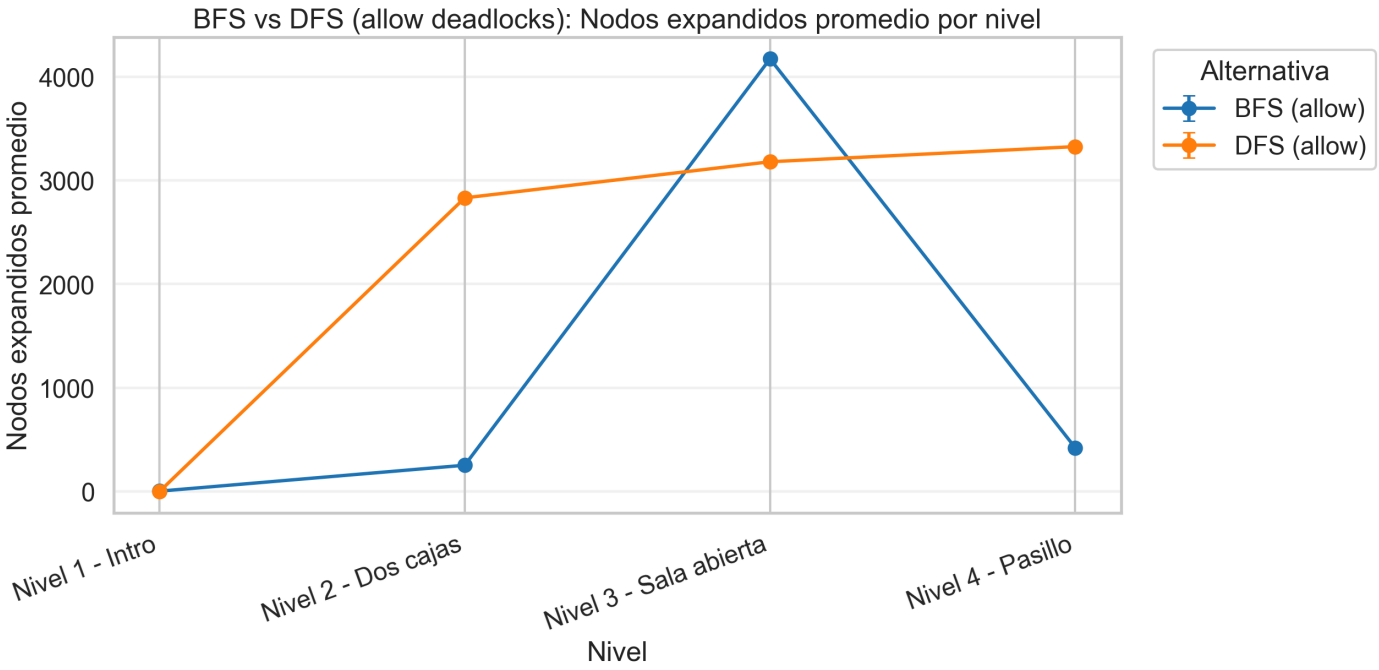


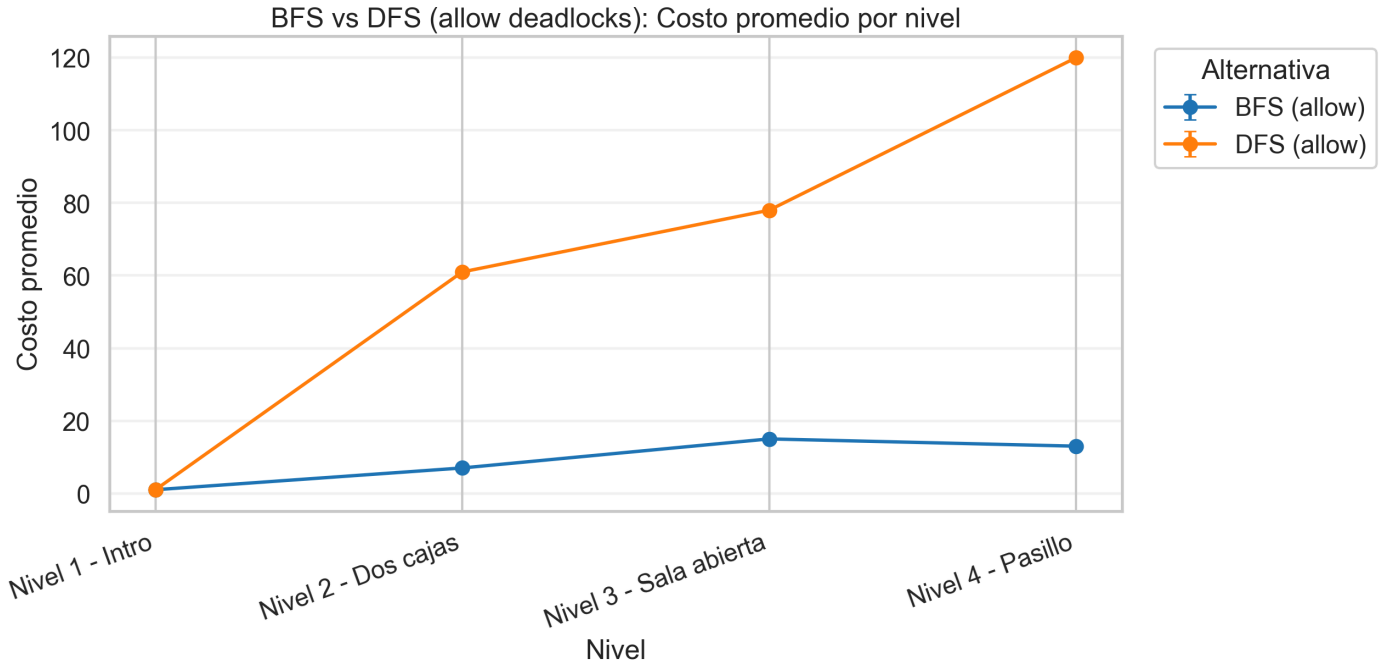




A* (Combined) optimiza la búsqueda reduciendo el espacio y tiempo en más de un **85% respecto a BFS**, demostrando que una heurística bien diseñada permite alcanzar la solución óptima explorando una fracción mínima del espacio de estados

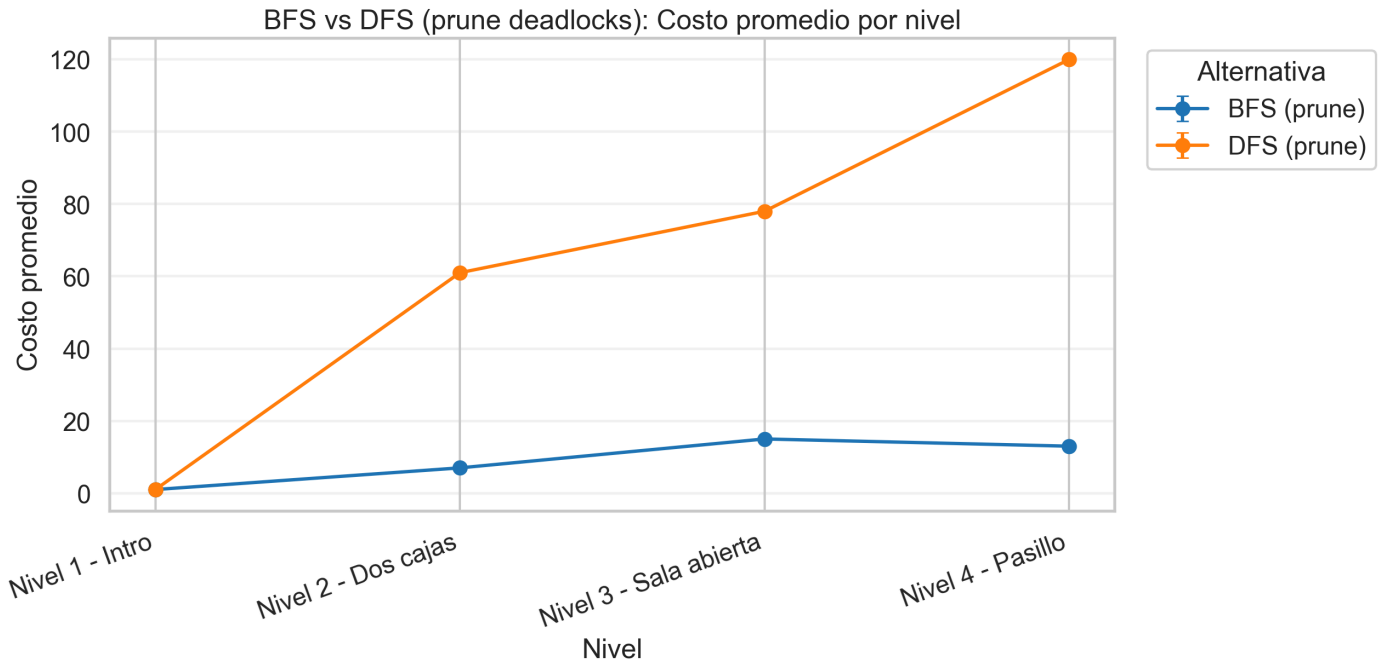
DFS vs BFS (allow) - nodos y costo





BFS vs DFS (prune) - costo y nodos expandidos

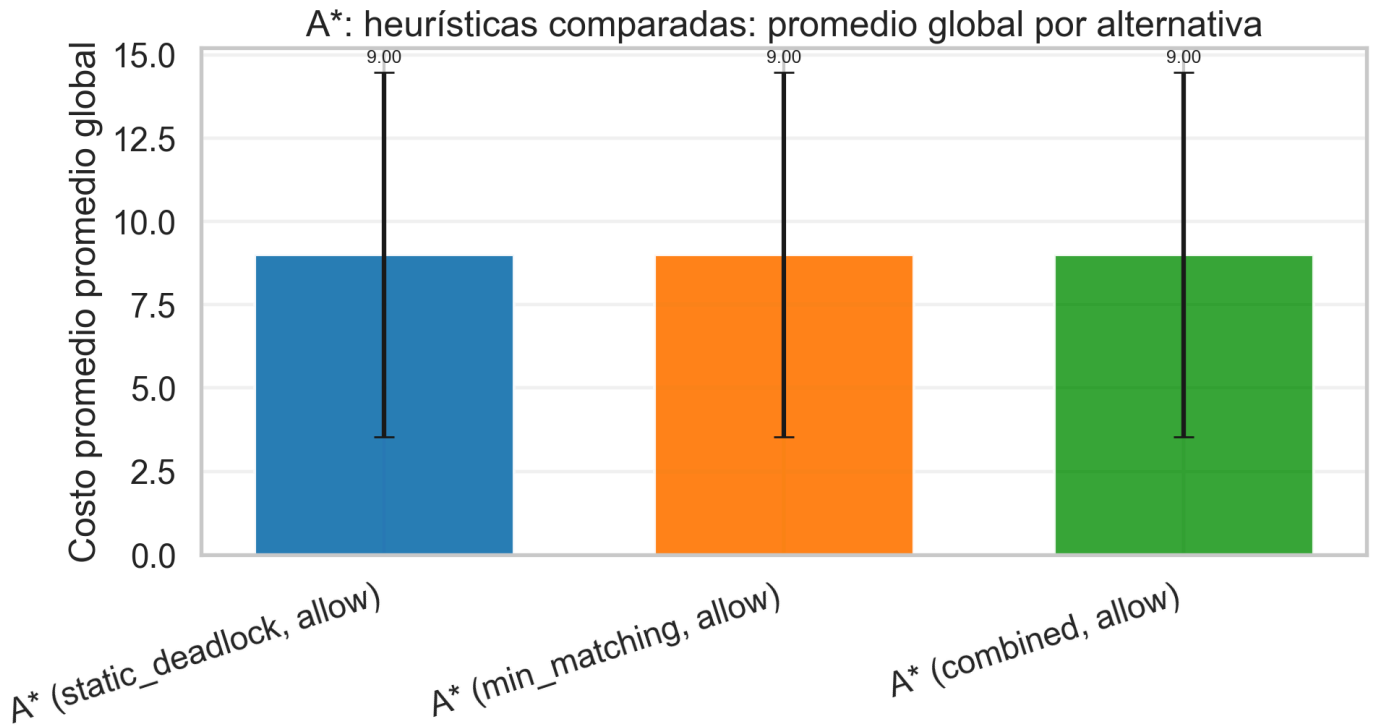
“Pasted image 20260325214329.png” could not be found.

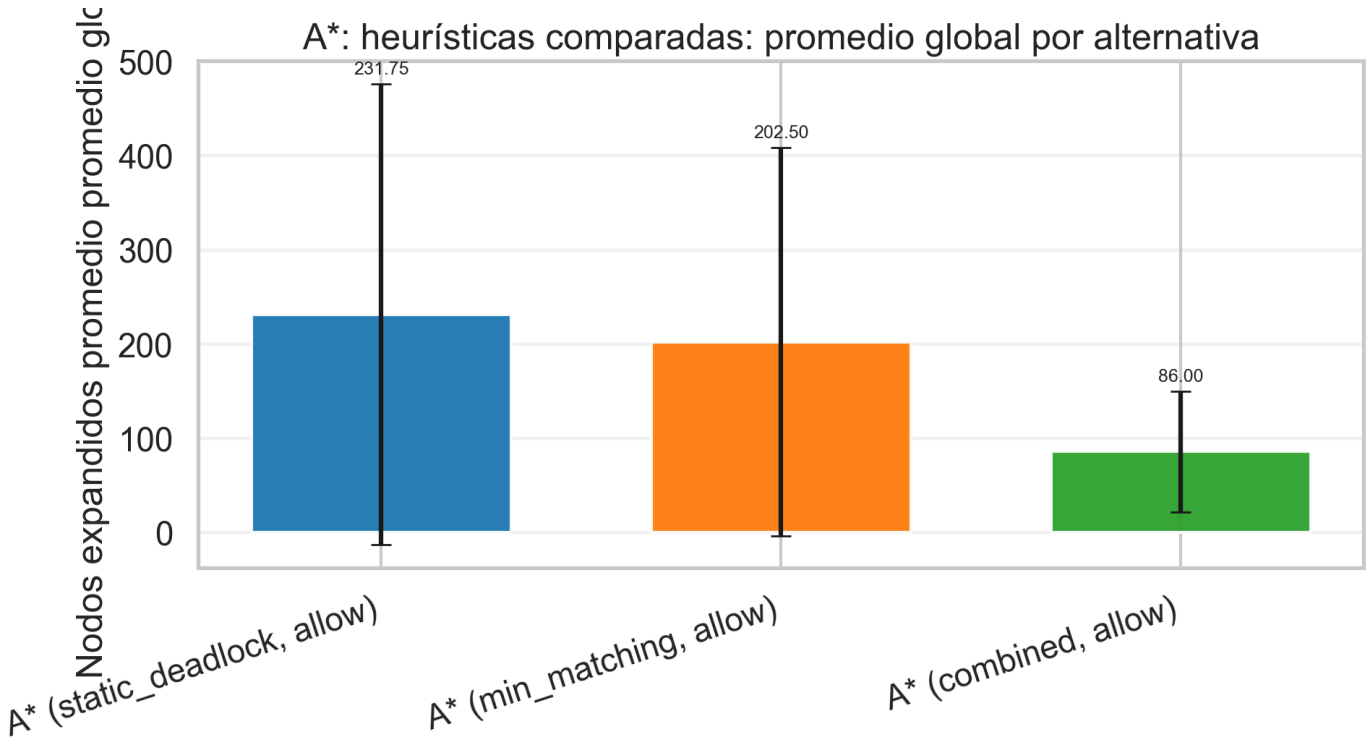


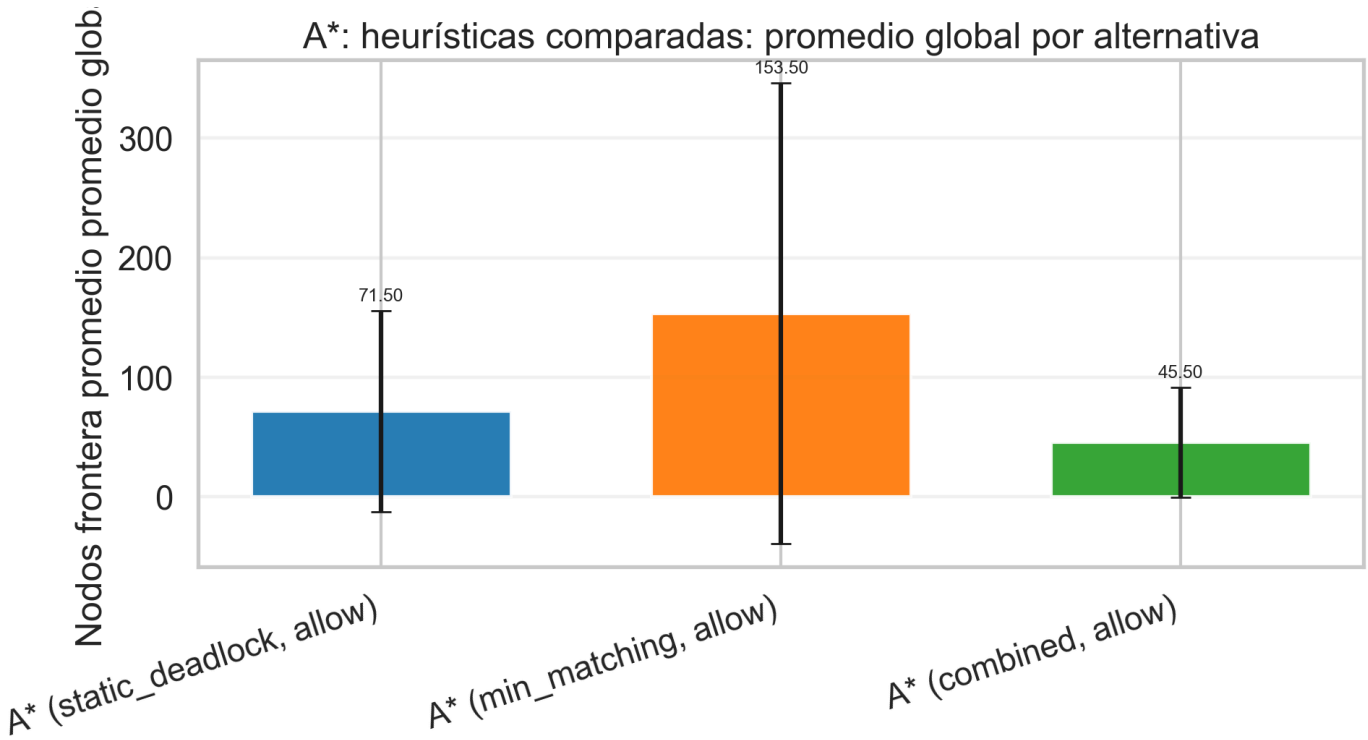
BFS vs. DFS: El impacto crítico de la poda de Deadlocks

- **Sin Poda (Allow): BFS domina en todo.** * DFS se pierde en ramas profundas e inútiles, expandiendo hasta **4000 nodos** en niveles abiertos.
  - BFS es más eficiente en nodos y drásticamente superior en costo (soluciones más cortas).
- **Con Poda (Prune): La falsa victoria de DFS.** * Al evitar estados de bloqueo, las expansiones de DFS caen en picada (de ~3000 a **<300**), llegando a expandir menos que BFS en niveles complejos como "Sala Abierta".
  - **La trampa:** Aunque DFS (prune) parece más "rápido" por expandir menos nodos, el gráfico de **Costo** revela que la calidad no mejora; DFS sigue entregando soluciones hasta 7 veces más largas que BFS.

HEURISITICAS A*







- **Consistencia en la Calidad (Costo = 9.00):** * La igualdad absoluta en los costos confirma que todas las heurísticas diseñadas son **admisibles**. Ninguna sobreestima el costo real, garantizando que el algoritmo siempre halle la solución óptima. El beneficio para el usuario es idéntico en todos los casos.
- **Eficiencia en la Exploración (Nodos y Frontera):** *Aquí es donde reside la verdadera diferencia. _A (Combined)_* demuestra ser la opción más robusta, reduciendo los nodos expandidos a solo **86.00** (una mejora del **63%** frente a la versión estática) y manteniendo la frontera más pequeña (**45.50**).
  - **El poder de la combinación:** Se observa que `Combined` no es simplemente una suma, sino una sinergia que logra podar el espacio de búsqueda de forma mucho más agresiva que `Static` o `Min_matching` por separado.
- **Análisis de Desempeño Relativo:**

- Mientras que `Static_deadlock` es la más "débil" en expansiones (231.75) y `Min_matching` la más pesada en frontera (153.50), la versión **Combined** logra el equilibrio perfecto: **mismo resultado óptimo con el mínimo esfuerzo computacional.**